# Current iteration — reproduction and consolidation

**Dates:** 2026-08-31 → 2026-09-01
**Status:** compute-blocked, two follow-ups outstanding
**Previous iteration:** §5n, the staged hybrid cascade (2026-08-29)
**Full record:** `HANDOVER.md` §5o. Per-architecture detail: `experiments/{baselines,eegnn,graph,cascade}/REPORT.md`

## What this iteration was

Not new research. Every number in `HANDOVER.md` had been produced by a single line of work, and the project had already been burned twice by numbers that did not survive re-running — §5f's "GCN/GAT fall below logistic regression" (refuted once seeds were added) and §5g's "37.0% depth saved" (one favourable seed; the 5-seed mean was 20.4% ± 13.6).

So: four agents, one architecture each, isolated folders, told to re-run from scratch and check whether the record holds. Explicitly *not* to tune, improve, or explore. Every claim came back marked REPRODUCED / REPRODUCED-WEAKER / NOT REPRODUCED / UNTESTABLE, with the re-measured number quoted next to the claimed one.

## The verdict

**The thing the project rests on is real. The things it abandoned deserved abandoning. Four recorded claims were wrong.**

### The pivot holds

The §5n result — that cascading over **information** (bedside → chart → labs) produces differentiated stages where cascading over **depth** did not — is the project's load-bearing claim. It reproduced:

| | recorded | re-measured |
|---|---|---|
| bedside / chart / labs AUROC | 0.8624 / 0.8733 / 0.8968 | 0.8623 / 0.8753 / 0.8974 |
| labs – bedside | +0.0343, t = 17.04 | **+0.0351, t = 14.76** |
| same sign across seeds | — | 5/5 |

Against the depth cascade's L1 – L10 gap of +0.0011 (t = 0.75), also reproduced. The contrast that motivated the pivot survives someone else re-running it.

The cohort itself reproduces **to the patient**: all six filter-ladder steps identical, prevalence 0.10780, 331 → 542 features, deterministic baselines matching to 4 decimal places.

## The graph negative hardened

Already a measured negative; now a stronger one. Five seeds (the recorded table used three), paired against the same run with the graph switched off — best variant **+0.0005, t = 0.76, not significant**, and both parameter-free variants are *negative* under honest hyper-parameter selection.

The sharp version: sweeping the correction strength and letting it **cheat by selecting on the test set** picks a strength of **zero in 4 of 5 seeds**, with an oracle ceiling of +0.0003. This is no longer "the graph does not earn its place". It is **no achievable setting of this stage helps, including one chosen with the answers in hand.** That is publishable as a negative result.

Separately, §7's long-standing multi-view fusion question closed the same way: the 0.811 → 0.846 gain is *entirely edge count*, and at matched density fusion loses at every density tested.

## Four corrections

1. **A real code bug.** `scripts/run_cascade.py:76` caches the kNN graph without the seed in the key, while `X` is rebuilt per seed. Runs 2–5 silently reuse run 1's graph — proved by hashing the edge lists (identical across all five seeds for kNN; correctly distinct for leaf). Understates variance; does not change the conclusion.
2. **A metric bug.** `src/evaluate.py:243` uses strict `>` where a tie is legal, so a gap of exactly 0.00 reports as an ordering *inversion*. One recorded

"0/3 ordering OK" failure is entirely this artefact.

3. **A statistic averaged over 2 seeds, printed as 5.** The α=0.01 row of the trade-off table gives "10.8% saved, miss rate 0.0074 ± 0.0011". The release side actually fired in 2 of 5 seeds; the miss rate is an n=2 average; and 10.8% is the mean of a bimodal quantity no seed is near. The Clopper–Pearson bound needs m ≥ 299 clean releases at α=0.01 and the ranking supplies at most 220 — the same arithmetic the record already applies at α=0.005, just never carried across. **α=0.005 and α=0.02 are unaffected; the curve between them is real.**
4. **"The sick exit earlier" is not reliable.** Recorded as −0.32 on one seed. Across five: −0.216, then +0.597 / +0.627 / +0.792 / +0.649. Mean +0.49 ± 0.40, t = +2.728 against critical 2.776. The **third** single-seed number in this project not to survive more seeds.

One further mechanism correction: §5j blamed the 0% depth saving on an oversized budget. The actual cause is that the halt head is saturated shut by the default loss weights ( `high_risk_halt_penalty` 5× `ponder_cost` ) — at budget 4 with ponder 0.05, early exit *does* engage, saving 56.9% at no accuracy cost. This does not rescue the depth architecture (the best budget-2 config is still "exit everyone at layer 1"), so the pivot stands, but the stated reason was wrong.

# What this changes about how to talk about the project

- **Lead with the staged cascade.** It is the one claim that has now been independently re-derived.
- **Sell the graph result as a negative finding**, not as a caveat. The oracle argument makes it much stronger than the recorded framing.
- **Drop α=0.01 from the trade-off curve**, or report it as "fires in 2/5 seeds".
- **Stop quoting single-seed numbers.** Three have now failed. Anything without a ± over ≥5 seeds should be treated as provisional.

# Method notes worth keeping

- Four agents against **one serialised Jupyter kernel** required: separate local folders (full repo copies), separate remote roots via a new `--root` flag on `scripts/remote_kernel.py` , one shared read-only cohort cache built once (414 s rather than four times), GPUs split 0/1, XGBoost pinned to 2 threads

each, and a mandatory detached-launch-and-poll pattern so no long job starved the others.
- **Thread pinning perturbs numerics.** XGBoost's histogram reduction order is thread-count dependent, so this pass is not bit-identical to the record — per-stage AUROCs drift 0.0001–0.0020. Irrelevant to every headline claim, decisive for the knife-edge α=0.01 row. Wall-clock `cost_us` is not comparable across the two settings at all; ratios are.
- Two agents died on an API session limit **after finishing all compute**. Their reports were reconstructed from raw JSON with every number re-derived. Their final in-flight sentences were work-in-progress, not findings, and were not treated as results.

# Outstanding — blocked on compute

The Kaggle session ended and took `/kaggle/working/shared_cache` with it. Both items have code written and waiting in the experiment folders:

1. **`--per-seed-graph` rerun** of the four graph variants, to quantify the cache bug. Expected to widen error bars on two rows without moving the null.
2. **Tie-aware certificate scan.** Isotonic regression produces flat regions, so a threshold landing on a tie block releases the whole block at once — a plausible route to clearing m ≥ 299 in a single jump, and the most likely full explanation of the α=0.01 behaviour. **A hypothesis with a test attached, not a finding.** Also unrun: α=0.05 and the `--ablate-conformal` arm.

Neither is load-bearing for any current claim. To resume: restart the Kaggle notebook, export a fresh `KERNEL_URL`, rebuild the cohort (~7 min, expect 17,663 patients / 542 features / 10.78% prevalence), then run the two items above.

# Next iteration should probably

Fix the two bugs in canonical, demote the α=0.01 row, and then decide whether the project is *finished as a negative-plus-one-positive result* — which it arguably is — or whether the staged cascade is worth extending to survival/readmission. The graph line of enquiry is closed; do not reopen it without a genuinely new edge source.